

HHR Solutions - Arquivo Criminal Brasileiro

HHR SOLUTIONS

Arquivo Criminal Brasileiro

ATV02 - API REST + MongoDB

Material de orientação. Define o que deve ser construído e onde consultar exemplos, sem entregar a implementação pronta.

GUIA 01 - BACKEND

1. Responsabilidade

Construir a API REST seguindo a organização estudada em aula:



Também será responsável por integrar autenticação, Argon2id, JWT, TMDB e a infraestrutura do Swagger.

O guia informa o contrato esperado. A lógica de cada função deve ser desenvolvida pelo responsável.

Referencia para esta secao

Aula 05 - estrutura geral da API: <https://github.com/haimonvieira/desenvolvimento-de-software-multiplataforma/tree/main/DSM3/DW3/dw3-api-express-mongodb/aula-05-hash-de-senha-e-variaveis-de-ambiente>

O que procurar: Observe a separação entre rotas, controllers, services, models, usuário e middleware.

Express - roteamento: <https://expressjs.com/pt-br/guide/routing/>

O que procurar: Use para entender método HTTP, caminho e handler sem copiar uma solução pronta.

Use a referencia para entender o padrao e desenvolver a propria implementacao/solucao.

2. Estrutura esperada

O projeto deve possuir uma organização equivalente à usada nas aulas.

Área	Arquivos esperados
entrada da aplicação	<code>index.js</code>
rotas	<code>casoRoutes.js</code> , <code>userRoutes.js</code> , <code>tmdbRoutes.js</code>
controllers	<code>casoController.js</code> , <code>UserController.js</code> , <code>tmdbController.js</code>
services	<code>casoService.js</code> , <code>userService.js</code> , <code>tmdbService.js</code>
models	<code>Casos.js</code> , <code>Users.js</code>
autenticação	<code>middleware/Auth.js</code>
banco	<code>config/db-connection.js</code>
Swagger	<code>config/swagger-config.js</code>
documentação Swagger	consumir <code>docs/swaggerDocs.yaml</code> criado pela Documentação

Referencia para esta secao

Aula 05 - organização de arquivos: <https://github.com/haimonvieira/desenvolvimento-de-software-multiplataforma/tree/main/DSM3/DW3/dw3-api-express-mongodb/aula-05-hash-de-senha-e-variaveis-de-ambiente>

O que procurar: Compare a estrutura de Games e Users com a estrutura esperada neste projeto.

Use a referencia para entender o padrao e desenvolver a propria implementacao/solucao.

3. Dependências do projeto

O Backend deve utilizar as mesmas tecnologias centrais das aulas e as extensões escolhidas pelo grupo.

Dependência	Uso
Express	API e rotas
Mongoose	comunicação com MongoDB
MongoDB	validação/utilidades usadas no padrão da aula

Dependência	Uso
dotenv	variáveis de ambiente
jsonwebtoken	geração e validação de JWT
argon2	hash Argon2id e verificação de senha
swagger-jsdoc	leitura da especificação Swagger/OpenAPI
swagger-ui-express	interface <code>/api-docs</code>
nodemon	execução durante desenvolvimento

Referencia para esta secao

Aula 05 - package.json: <https://github.com/haimonvieira/desenvolvimento-de-software-multiplataforma/tree/main/DSM3/DW3/dw3-api-express-mongodb/aula-05-hash-de-senha-e-variaveis-de-ambiente>

O que procurar: Use a lista de dependências da aula como base.

node-argon2: <https://github.com/ransalt/node-argon2>

O que procurar: Consulte a documentação da biblioteca adotada no lugar do bcrypt.

Swagger do professor: https://github.com/maxxdiego/17_documentacao_de_api_com_swagger

O que procurar: Consulte as dependências usadas para publicar o Swagger UI.

Use a referencia para entender o padrao e desenvolver a propria implementacao/solucao.

4. Variáveis de ambiente

O Backend deve prever:

Variável	Finalidade
PORT	porta da API
JWT_SECRET	segredo usado pelo JWT
TMDB_API_TOKEN	token da API TMDB
MONGODB_USERNAME	usuário do Atlas
MONGODB_PASSWORD	senha do Atlas

O padrão da Aula 03.2 pode ser usado como referência para separar as credenciais do Atlas.

Referencia para esta secao

Aula 03.2 - MongoDB Atlas e variáveis: <https://github.com/haimonvieira/desenvolvimento-de-software-multiplataforma/tree/main/DSM3/DW3/dw3-api-express-mongodb/aula-03.2-hospedagem-no-mongodb-atlas>

O que procurar: Observe como as credenciais do Atlas são lidas por variáveis de ambiente.

Aula 05 - JWT_SECRET: [https://github.com/haimonvieira/ desenvolvimento-de-software-multiplataforma/tree/main/DSM3/DW3/dw3-api-express-mongodb/aula-05-hash-de-senha-e-variaveis-de-ambiente](https://github.com/haimonvieira/desenvolvimento-de-software-multiplataforma/tree/main/DSM3/DW3/dw3-api-express-mongodb/aula-05-hash-de-senha-e-variaveis-de-ambiente)

O que procurar: Observe onde o segredo JWT é lido no fluxo de autenticação.

Use a referencia para entender o padrao e desenvolver a propria implementacao/solucao.

5. Contrato de autenticação

Cadastro

Item	Definição
Método	POST
Rota	/user
Autenticação	pública
Entrada	email, password
Processo esperado	gerar hash Argon2id e persistir usuário
Sucesso	201
Falha geral	500

A senha recebida deve ser transformada em hash antes da persistência.

Referência de fluxo semelhante: `UserController.js` da Aula 05.
Troca definida pelo grupo: `bcrypt` da aula por `Argon2id`.

Login

Item	Definição
Método	POST
Rota	/login
Autenticação	pública
Entrada	email, password

Item	Definição
Busca	localizar usuário pelo e-mail
Verificação	comparar senha com hash Argon2id
Saída de sucesso	token JWT
Expiração de referência	48 horas, seguindo a Aula 05
Possíveis status	200, 400, 401, 404, 500

Middleware

Criar um middleware equivalente ao `Auth.Authorization` da Aula 05.

Ele deve:

1. ler o cabeçalho `Authorization`;
2. trabalhar com o formato `Bearer TOKEN`;
3. validar o JWT;
4. liberar a requisição quando o token estiver válido;
5. responder com 401 quando a autenticação falhar.

Referencia para esta secao

Aula 05 - userController e Auth: <https://github.com/haimonvieira/desenvolvimento-de-software-multiplataforma/tree/main/DSM3/DW3/dw3-api-express-mongodb/aula-05-hash-de-senha-e-variaveis-de-ambiente>

O que procurar: Use como referência do fluxo cadastro, login, JWT e middleware.

node-argon2: <https://github.com/ransalt/node-argon2>

O que procurar: Pesquise hash e verify para implementar a troca de bcrypt por Argon2id.

jsonwebtoken: <https://github.com/auth0/node-jsonwebtoken/blob/master/README.md>

O que procurar: Consulte sign e verify.

Use a referencia para entender o padrao e desenvolver a propria implementacao/solucao.

6. Contrato das rotas de Caso

Todas as rotas abaixo são protegidas por Bearer Token.

GET /casos

Item	Definição
Finalidade	listar todos os casos
Entrada	nenhuma
Retorno	coleção/lista de casos
Sucesso	200
Erro geral	500

POST /casos

Item	Definição
Finalidade	cadastrar um caso
Body	estrutura completa de Caso
Sucesso	201
Erro geral	500

Campos esperados no Body:



Significado dos campos que podem gerar dúvida:

- `categorias[]` : lista de classificações do caso. Pode receber mais de um texto.
- `detalhes` : objeto aninhado com informações gerais do próprio caso.
- `situacaoJudicial` : texto simples com a situação judicial geral registrada para o caso.
- `producoes[]` : lista de filmes, séries ou documentários relacionados. Cada item segue o formato de Produção definido pelo Banco.

- `_id`: não deve vir no Body. O MongoDB gera esse identificador automaticamente quando o documento é salvo.

GET `/casos/:id`

Item	Definição
Finalidade	consultar um caso específico
Parâmetro	<code>id</code>
Validação	ObjectId válido
Sucesso	200
ID inválido	400
Caso inexistente	404
Erro geral	500

PUT `/casos/:id`

Item	Definição
Finalidade	atualizar um caso
Parâmetro	<code>id</code>
Body	campos do Caso
Validação	ObjectId válido
Sucesso	200
ID inválido	400
Erro geral	500

DELETE `/casos/:id`

Item	Definição
Finalidade	excluir um caso
Parâmetro	<code>id</code>
Validação	ObjectId válido
Sucesso	204
ID inválido	400
Erro geral	500

Referencia para esta secao

Aula 05 - CRUD de Games: <https://github.com/haimonvieira/desenvolvimento-de-software-multiplataforma/tree/main/DSM3/DW3/dw3-api-express-mongodb/aula-05-hash-de-senha-e-variaveis-de-ambiente>

O que procurar: Use gameRoutes, gameController e gameService como padrão para estruturar o CRUD de Caso.

Express - roteamento: <https://expressjs.com/pt-br/guide/routing/>

O que procurar: Consulte parâmetros de rota como :id e métodos GET, POST, PUT e DELETE.

Use a referencia para entender o padrao e desenvolver a propria implementacao/solucao.

7. Organização Controller, Service e Model

Seguir o padrão dos Games da aula.

Route

Responsável por associar:



Controller

Responsável por:



Service

Responsável por:



Model

É fornecido pelo responsável do Banco e define a estrutura persistida.

Referência para essa separação: arquivos `gameRoutes.js`, `gameController.js` e `gameService.js` das aulas.

Referencia para esta secao

Aula 05 - Route, Controller e Service: <https://github.com/haimonvieira/desenvolvimento-de-software-multiplataforma/tree/main/DSM3/DW3/dw3-api-express-mongodb/aula-05-hash-de-senha-e-variaveis-de-ambiente>

O que procurar: Compare as responsabilidades de cada camada nos arquivos de Games.

Use a referencia para entender o padrao e desenvolver a propria implementacao/solucao.

8. Integração TMDb

A integração é uma extensão do projeto.

Rota

Item	Definição
Método	GET
Rota	<code>/tmdb/buscar</code>
Autenticação	Bearer Token
Query string	<code>query</code>
Exemplo de chamada	<code>/tmdb/buscar?query=Tremembé</code>
Origem externa	TMDb Search Multi
Sucesso	200
Erro geral	500

Resultado que interessa ao projeto

Normalizar cada produção para:

Campo	Tipo
<code>tmdbId</code>	Number
<code>titulo</code>	String
<code>tipo</code>	String
<code>ano</code>	Number
<code>sinopse</code>	String

Campo	Tipo
poster	String

A busca apenas retorna opções para auxiliar o preenchimento. Quando o usuário escolher uma produção, os campos usados pelo projeto podem ser aproveitados no cadastro ou atualização de `producoes[]` dentro do Caso. A busca TMDB não cria uma entidade ou collection separada.

Referência: <https://developer.themoviedb.org/reference/search-multi>

Referencia para esta secao

TMDB - Search Multi: <https://developer.themoviedb.org/reference/search-multi>

O que procurar: Consulte parâmetros, autenticação da API e estrutura do resultado da pesquisa.

Use a referencia para entender o padrao e desenvolver a propria implementacao/solucao.

9. Swagger - responsabilidade do Backend

O Backend é responsável por fazer a infraestrutura do Swagger funcionar.

Deve:

1. adicionar as dependências de Swagger usadas no repositório do professor;
2. criar a configuração equivalente a `swagger-config.js`;
3. apontar a configuração para `docs/swaggerDocs.yaml`;
4. integrar o Swagger ao Express;
5. disponibilizar a interface em `/api-docs`;
6. configurar suporte a Bearer/JWT.

A pessoa de Documentação é responsável pelo conteúdo do `swaggerDocs.yaml`.

Referência: https://github.com/maxxdiego/17_documentacao_de_api_com_swagger

Referencia para esta secao

Swagger do professor: https://github.com/maxxdiego/17_documentacao_de_api_com_swagger

O que procurar: Siga o padrão de `swagger-jsdoc`, `swagger-ui-express`, `swagger-config.js` e `/api-docs`.

Swagger - Bearer Authentication: https://swagger.io/docs/specification/v3_0/authentication/bearer-authentication/

O que procurar: Use para configurar/documentar o esquema Bearer JWT.
Use a referencia para entender o padrao e desenvolver a propria implementacao/solucao.

10. Dependência do Banco

O Backend pode iniciar rotas, controllers, services, autenticação, TMDB e Swagger antes do Banco terminar.

Para realizar os testes reais de persistência, deve receber:



Referencia para esta secao

Aula 03.2 - conexão Atlas: <https://github.com/haimonvieira/desenvolvimento-de-software-multiplataforma/tree/main/DSM3/DW3/dw3-api-express-mongodb/aula-03.2-hospedagem-no-mongodb-atlas>

O que procurar: Mostra o arquivo de conexão que o Banco precisa fornecer.

Aula 03.1 - Model aninhado: <https://github.com/haimonvieira/desenvolvimento-de-software-multiplataforma/tree/main/DSM3/DW3/dw3-api-express-mongodb/aula-03.1-documentos-aninhados>

O que procurar: Mostra o tipo de Model que o Backend consumirá.
Use a referencia para entender o padrao e desenvolver a propria implementacao/solucao.

11. Entregas para a Documentação

Quando as rotas estiverem definidas, informar:

- método HTTP;
- rota;
- se é pública ou protegida;
- parâmetros;
- query strings;
- campos do Body;
- formato geral da resposta;
- códigos HTTP utilizados.

A Documentação usa isso para Swagger, README e Insomnia.

Referencia para esta secao

Swagger do professor: https://github.com/maxxdiego/17_documentacao_de_api_com_swagger

O que procurar: Ajuda a entender quais informações de rota a Documentação precisa receber.

OpenAPI - Paths and Operations: https://swagger.io/docs/specification/v3_0/paths-and-operations/

O que procurar: Use como checklist de método, caminho, parâmetros e respostas.

Use a referencia para entender o padrao e desenvolver a propria implementacao/solucao.

12. Critério de conclusão

O Backend está concluído quando for possível demonstrar:



Referencia para esta secao

Insomnia - Requests: <https://developer.konghq.com/insomnia/requests/>

O que procurar: Use como referência para validar cada operação antes de considerar o Backend concluído.

Insomnia - autenticação: <https://developer.konghq.com/insomnia/request-authentication/>

O que procurar: Use para testar Bearer Token nas rotas protegidas. Use a referencia para entender o padrao e desenvolver a propria implementacao/solucao.

13. Onde estudar para implementar

Estrutura Route -> Controller -> Service -> Model

<https://github.com/haimonvieira/desenvolvimento-de-software-multiplataforma/tree/main/DSM3/DW3/dw3-api-express-mongodb/aula-05-hash-de-senha-e-variaveis-de-ambiente>

Consultar principalmente:



Usuário e JWT

<https://github.com/haimonvieira/desenvolvimento-de-software-multiplataforma/tree/main/DSM3/DW3/dw3-api-express-mongodb/aula-05-hash-de-senha-e-variaveis-de-ambiente>

Consultar:



MongoDB Atlas

<https://github.com/haimonvieira/desenvolvimento-de-software-multiplataforma/tree/main/DSM3/DW3/dw3-api-express-mongodb/aula-03.2-hospedagem-no-mongodb-atlas>

Argon2id

<https://github.com/ransalt/node-argon2>

TMDB

<https://developer.themoviedb.org/reference/search-multi>

Swagger

https://github.com/maxxdiego/17_documentacao_de_api_com_swagger

Referencia para esta secao

Aula 05: <https://github.com/haimonvieira/desenvolvimento-de-software-multiplataforma/tree/main/DSM3/DW3/dw3-api-express-mongodb/aula-05-hash-de-senha-e-variaveis-de-ambiente>

O que procurar: Referência principal para o padrão da disciplina.

Aula 03.2: <https://github.com/haimonvieira/desenvolvimento-de-software-multiplataforma/tree/main/DSM3/DW3/dw3-api-express-mongodb/aula-03.2-hospedagem-no-mongodb-atlas>

O que procurar: Referência de Atlas.

Swagger do professor: https://github.com/maxxdiego/17_documentacao_de_api_com_swagger

O que procurar: Referência de documentação técnica.

Use a referencia para entender o padrao e desenvolver a propria implementacao/solucao.